

Parte 4 — Tuning, benchmark, calibração e limites

Tutorial completo do sciModelFlowR (versão 1.0.2)

Walter Esfrain Pereira

2026-09-19

Índice

1	Ajuste de hiperparâmetros sem otimismo	3
2	Benchmark: comparar modelos com critério declarado	7
3	Calibração: o número quer dizer o que promete?	9
4	Limiares: transformar probabilidade em decisão	14
5	Exercícios da Parte 4	16

1 Ajuste de hiperparâmetros sem otimismo

Duas perguntas diferentes: **qual configuração** do mesmo modelo é melhor (tuning) e **qual modelo** é melhor (benchmark). O pacote separa as duas e exige que o critério de decisão seja explícito — inclusive nos casos multiobjetivo.

```
campo <- ler("field_yield")
v <- verdade$field_yield
sem_out <- campo[!campo$plot_id %in% v$outlier_ids, ]

spec_xgb <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter",
    "block", "genotype"),
    id_column = "plot_id", categorical = c("block", "genotype")),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  model = smf_model_spec("boosted_tree", "xgboost",
    parameters = list(nrounds = 60L, params = list(max_depth = 3L))),
  metrics = list(smf_metric_spec("rmse", "minimize"), smf_metric_spec("mae", "minimize")),
  reproducibility = smf_reproducibility_spec(61L)
)
espaco <- smf_search_space(nrounds = smf_param_int(30L, 90L),
  params.max_depth = smf_param_int(2L, 6L))
print(espaco)
```

```
<sciModelFlowR::SearchSpace>
@ parameters:List of 2
.. $ nrounds      :List of 7
.. ..$ type       : chr "int"
.. ..$ lower      : int 30
.. ..$ upper      : int 90
.. ..$ values     : NULL
.. ..$ log        : logi FALSE
.. ..$ depends_on: NULL
.. ..$ resource   : logi FALSE
.. $ params.max_depth:List of 7
.. ..$ type       : chr "int"
.. ..$ lower      : int 2
.. ..$ upper      : int 6
.. ..$ values     : NULL
.. ..$ log        : logi FALSE
```

```

.. ..$ depends_on: NULL
.. ..$ resource : logi FALSE
@ conditions:List of 2
.. $ nrounds      : NULL
.. $ params.max_depth: NULL
@ metadata : list()

```

Quatro estratégias de busca, o mesmo espaço e o mesmo orçamento por configuração. O ponto pedagógico: **o método de busca é decisão de orçamento, não de ciência** — e o resample interno precisa ser idêntico para que as comparações façam sentido.

```

t_grid <- smf_tune(spec_xgb, sem_out, espaco, smf_tuning_spec("grid", budget = 4L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 3L, seed = 61L),
  objectives = list(smf_metric_spec("rmse", "minimize")), seed = 61L))
t_rand <- smf_tune(spec_xgb, sem_out, espaco, smf_tuning_spec("random", budget = 6L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 3L, seed = 62L),
  objectives = list(smf_metric_spec("rmse", "minimize"), smf_metric_spec("mae", "minimize")),
  decision_rule = list(type = "weighted_sum", weights = c(rmse = 0.7, mae = 0.3)),
  seed = 62L))
t_race <- smf_tune(spec_xgb, sem_out, espaco, smf_tuning_spec("racing", budget = 6L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 4L, seed = 63L),
  objectives = list(smf_metric_spec("rmse", "minimize")), seed = 63L,
  parameters = list(min_folds = 2L)))
t_bayes <- smf_tune(spec_xgb, sem_out, espaco, smf_tuning_spec("bayesian", budget = 6L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 3L, seed = 64L),
  objectives = list(smf_metric_spec("rmse", "minimize")), seed = 64L,
  parameters = list(initial = 3L, candidate_pool = 16L)))

tab_metodos <- data.frame(
  metodo = c("grid", "random", "racing", "bayesian"),
  avaliacoes = c(nrow(t_grid@archive), nrow(t_rand@archive),
    nrow(t_race@archive), nrow(t_bayes@archive)),
  melhor_rmse = c(min(t_grid@archive$rmse, na.rm = TRUE),
    min(t_rand@archive$rmse, na.rm = TRUE),
    min(t_race@archive$rmse, na.rm = TRUE),
    min(t_bayes@archive$rmse, na.rm = TRUE))
)
tab_smf(tab_metodos, caption = "Quatro estratégias de busca no mesmo espaço e com o mesmo orçamento")

```

Tabela 1.1: Quatro estratégias de busca no mesmo espaço e com o mesmo orçamento de folds internos.

metodo	avaliacoes	melhor_rmse
grid	4	457,446
random	6	435,995
racing	30	287,351
bayesian	6	453,169

```

tab_rand <- t_rand@archive[, c("trial", "nrounds", "params.max_depth", "rmse", "mae")]
tab_sel <- t_rand@selected[, c("nrounds", "params.max_depth", "rmse", "mae")]
tab_smf(tab_rand, caption = "Archive do random search: cada linha é uma configuração avaliada")

```

Tabela 1.2: Archive do random search: cada linha é uma configuração avaliada no resample interno.

trial	nrounds	params.max_depth	rmse	mae
1	57	4	450,784	350,966
2	34	6	466,550	364,083
3	59	6	466,766	364,138
4	82	5	477,875	364,123
5	33	2	435,995	330,821
6	36	6	466,505	364,089

```

tab_smf(tab_sel, caption = "Selecionada pela regra de compromisso declarada (pesos 0,7 para RMSE e 0,3 para MAE)")

```

Tabela 1.3: Selecionada pela regra de compromisso declarada (pesos 0,7 para RMSE e 0,3 para MAE).

nrounds	params.max_depth	rmse	mae
5	33	2	435,995
			330,821

```

arquivos <- rbind(
  data.frame(metodo = "grid", trial = t_grid@archive$trial, rmse = t_grid@archive$rmse),
  data.frame(metodo = "random", trial = t_rand@archive$trial, rmse = t_rand@archive$rmse),
  data.frame(metodo = "racing", trial = t_race@archive$trial, rmse = t_race@archive$rmse),
  data.frame(metodo = "bayesian", trial = t_bayes@archive$trial, rmse = t_bayes@archive$rmse)
)
ggplot(arquivos, aes(trial, rmse, colour = metodo)) +
  geom_point(size = 2, alpha = .8) +
  geom_hline(yintercept = min(arquivos$rmse, na.rm = TRUE), linetype = "dashed",
    colour = cores_smf[["alerta"]]) +
  facet_wrap(~metodo, nrow = 2) +
  scale_colour_manual(values = c(grid = cores_smf[["neutro"]], random = cores_smf[["primaria"]],
    racing = cores_smf[["destaque"]], bayesian = cores_smf[["sucesso"]]),
    guide = "none") +
  labs(x = "avaliação", y = "RMSE interno (kg/ha)",
    title = "Tuning archives: four search strategies")

```

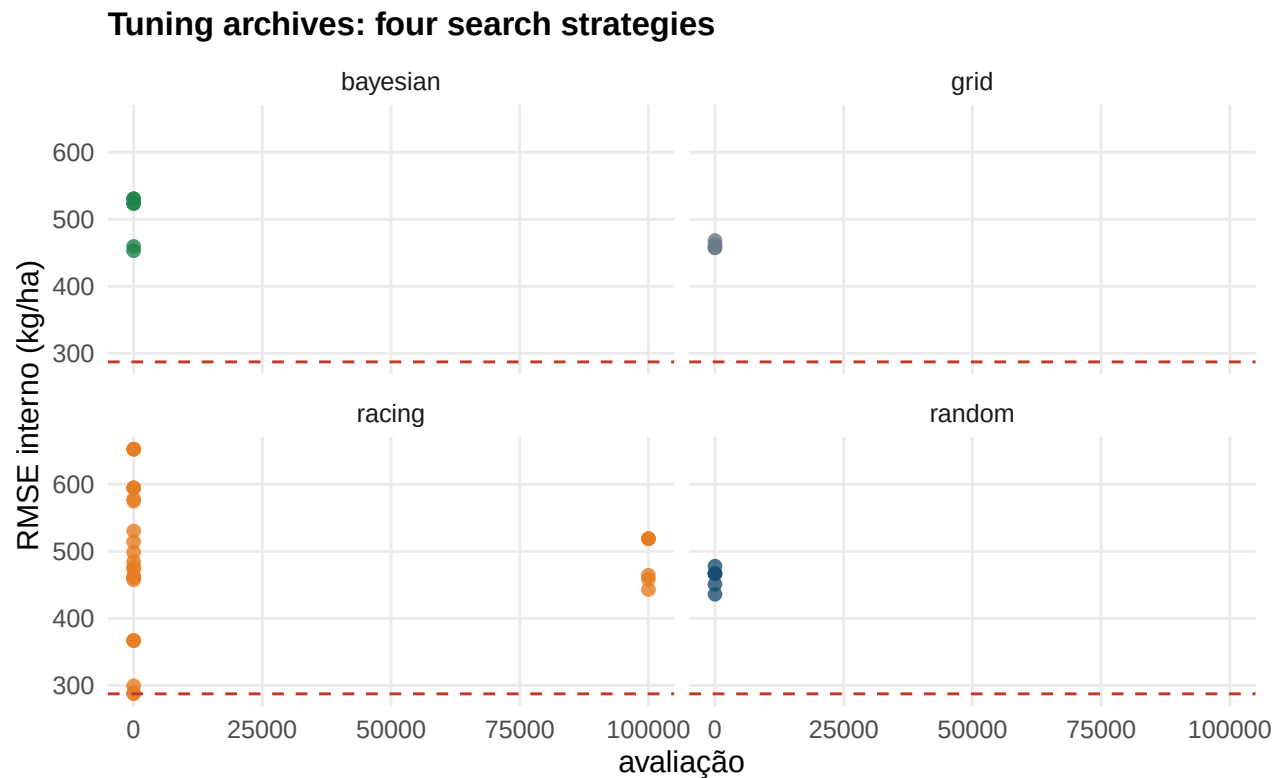


Figura 1.1: O archive completo: cada ponto é uma configuração; a linha tracejada marca a melhor média interna. O racing elimina candidatos fold a fold.

⚠ Armadilha

Um objetivo que **não** está em `spec@metrics` não produz archive vazio: ele aborta com erro claro (`TUNING_OBJECTIVE_NOT_IN_METRICS`), listando as métricas disponíveis. Esse comportamento é a correção de 1.0.2 de um defeito que devolvia seleção vazia em silêncio. O pacote prefere falhar alto.

i Interpretação

O melhor RMSE interno entre todos os métodos é 287,4 kg/ha, encontrado por racing. Os valores internos **não** são o desempenho final: eles escolhem a configuração. A estimativa honesta do escolhido vem de um resample externo (aqui, o holdout da Parte 3; no limite, tuning aninhado).

2 Benchmark: comparar modelos com critério declarado

O benchmark avalia vários **modelos** no mesmo desenho, com repetições e medidas de custo computacional. A decisão vem de uma regra explícita.

```
candidatos <- list(
  lm = smf_model_spec("linear_regression", "stats"),
  rf = smf_model_spec("random_forest", "tidymodels",
    parameters = list(model_engine = "ranger")),
  xgb = smf_model_spec("boosted_tree", "xgboost", parameters = list(nrounds = 60L))
)
bench <- smf_benchmark(spec_xgb, sem_out, smf_benchmark_spec(
  candidates = candidatos,
  metrics = list(smf_metric_spec("rmse", "minimize")),
  resampling = smf_resampling_spec("kfold", n_splits = 5L, seed = 71L),
  compute_measures = c("fit_time", "total_time"),
  repetitions = 2L, seed = 71L
))
tab_smf(bench@summary, caption = "Benchmark com 5 folds repetidos 2 vezes: média, desvio, erro-padrão e intervalo por candidato")
```

Tabela 2.1: Benchmark com 5 folds repetidos 2 vezes: média, desvio, erro-padrão e intervalo por candidato.

candidate	metric	mean	sd	se	lower	upper	n
lm	rmse	373,236	138,221	43,709	274,358	472,113	10
rf	rmse	584,586	82,826	26,192	525,336	643,836	10
xgb	rmse	497,491	75,344	23,826	443,594	551,389	10

```
dec <- smf_decide_benchmark(bench, list(type = "weighted_sum", weights = c(rmse = 1)))
tab_smf(dec, caption = "Decisão pela regra declarada (peso 1 para RMSE).")
```

Tabela 2.2: Decisão pela regra declarada (peso 1 para RMSE).

candidate	rmse	.decision_score
lm	373,236	0

```
ggplot(bench@summary, aes(stats::reorder(candidate, mean), mean)) +
  geom_pointrange(aes(ymin = lower, ymax = upper), colour = cores_smf[["primaria"]], size = .) +
  geom_text(aes(label = fmt(mean, 0)), hjust = -0.35) +
  coord_flip() +
  labs(x = NULL, y = "RMSE médio (kg/ha)",
       title = "Benchmark: mean RMSE with interval") +
  ylim(0, max(bench@summary$upper) * 1.15)
```

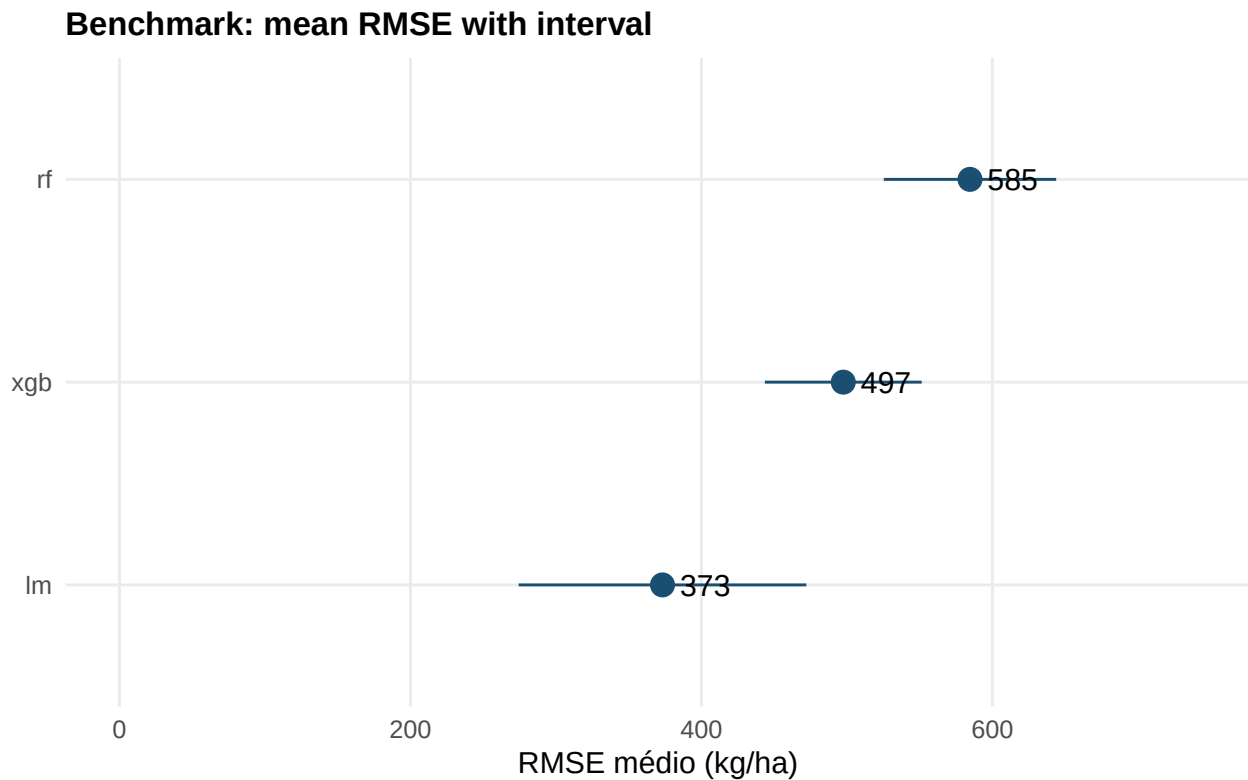


Figura 2.1: Desempenho médio com intervalo de confiança por candidato: a reta vence as alternativas flexíveis neste problema linear.

i Interpretação

O vencedor pela regra é lm, com RMSE médio 373,2. Os intervalos se sobrepõem em parte, mas a conclusão não depende de um único fold: ela vem de 10 avaliações (5 folds $\times$ 2 repetições) e de uma regra declarada antes de olhar o resultado.

3 Calibração: o número quer dizer o que promete?

Uma probabilidade de 0,3 deve corresponder a ~30% de eventos. No conjunto de doença, com ruído de rótulo, isso raramente acontece sem calibração. O pacote oferece o calibrador (`smf_calibrate()`), a aplicação (`smf_apply_calibration()`) e o relatório (`smf_calibration_report()`).

```
doenca <- ler("disease_binary")
doenca$disease <- factor(doenca$disease, levels = c(0, 1))
set.seed(20260919)
idx <- sample.int(nrow(doenca))
i_te <- idx[1:100]
i_resto <- idx[101:nrow(doenca)]
i_cal <- i_resto[1:120]
i_tr <- setdiff(i_resto, i_cal)
glm_p4 <- stats::glm(disease ~ x1 + x2 + x3 + x4, data = doenca[i_tr, ],
                    family = stats::binomial())
p_cal_tr <- stats::predict(glm_p4, doenca[i_cal, ], type = "response")
p_te <- stats::predict(glm_p4, doenca[i_te, ], type = "response")
P_te <- cbind(`0` = 1 - p_te, `1` = p_te)
cal_p4 <- smf_calibrate(doenca$disease[i_cal],
                      cbind(`0` = 1 - p_cal_tr, `1` = p_cal_tr),
                      smf_calibration_spec("platt"))
P_te_cal <- smf_apply_calibration(cal_p4, P_te)
rel_raw <- smf_calibration_report(doenca$disease[i_te], P_te, bins = 5L,
                                positive_label = "1")
rel_cal <- smf_calibration_report(doenca$disease[i_te], P_te_cal, bins = 5L,
                                positive_label = "1")
tab_cal <- data.frame(
  medida = c("ECE", "log loss", "Brier"),
  bruto = c(rel_raw$ece, rel_raw$log_loss, rel_raw$brier),
  calibrado = c(rel_cal$ece, rel_cal$log_loss, rel_cal$brier)
)
tab_smf(tab_cal, caption = "Mesmo teste, mesma logística: antes e depois da calibração Platt")
```

Tabela 3.1: Mesmo teste, mesma logística: antes e depois da calibração Platt (partição de calibração: 120 linhas).

medida	bruto	calibrado
ECE	0,057	0,049
log loss	0,424	0,419

medida	bruto	calibrado
Brier	0,236	0,235

```

conf_raw <- rel_raw$table
conf_raw <- data.frame(previsto = conf_raw$mean_probability,
                       observado = conf_raw$observed_frequency,
                       versao = "bruto")
conf_cal <- rel_cal$table
conf_cal <- data.frame(previsto = conf_cal$mean_probability,
                       observado = conf_cal$observed_frequency,
                       versao = "calibrado")
ggplot(rbind(conf_raw, conf_cal), aes(previsto, observado, colour = versao)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey55") +
  geom_point(size = 2.6) +
  geom_line(alpha = .6) +
  scale_colour_manual(values = c(bruto = cores_smf[["neutro"]],
                                calibrado = cores_smf[["primaria"]])) +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(x = "probabilidade prevista (média do bin)", y = "frequência observada",
       colour = NULL,
       title = "Reliability curve: raw vs calibrated") +
  theme(legend.position = "top")

```

Reliability curve: raw vs calibrated

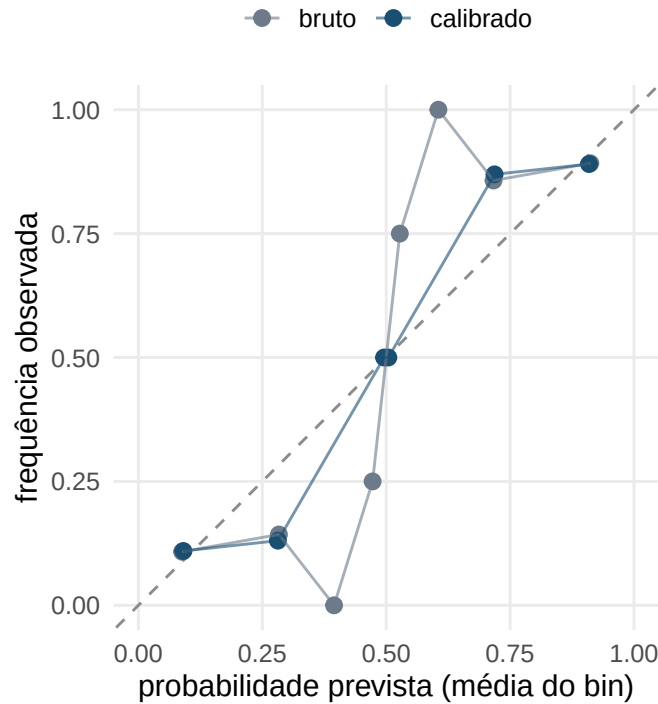


Figura 3.1: Curva de confiabilidade: probabilidade prevista contra frequência observada. A diagonal é a calibração perfeita.

! Interpretação

Nesta partição, o ECE bruto é 0,0571 e o calibrado é 0,0486. Calibrar **não** é automaticamente melhor: com apenas 120 linhas de calibração, o calibrador Platt pode piorar o ECE — o efeito depende do tamanho da partição e do ruído. A taxa abaixo mede o procedimento com repetição, que é a única evidência que decide.

```
uma_replicata_cal <- function(s, n_cal) {  
  set.seed(s)  
  n <- nrow(doenca); ordem <- sample.int(n)  
  i_te_r <- ordem[1:100]  
  resto <- ordem[101:n]  
  i_cal_r <- resto[1:n_cal]  
  i_tr_r <- setdiff(resto, i_cal_r)  
  f <- stats::glm(disease ~ x1 + x2 + x3 + x4, data = doenca[i_tr_r, ],  
                 family = stats::binomial())  
  p_c <- stats::predict(f, doenca[i_cal_r, ], type = "response")  
  p_t <- stats::predict(f, doenca[i_te_r, ], type = "response")  
  Pc <- cbind(`0` = 1 - p_c, `1` = p_c)  
  Pt <- cbind(`0` = 1 - p_t, `1` = p_t)  
  cr <- smf_calibrate(doenca$disease[i_cal_r], Pc, smf_calibration_spec("platt"))
```

```

Ptc <- smf_apply_calibration(cr, Pt)
r0 <- smf_calibration_report(doenca$disease[i_te_r], Pt, bins = 5L, positive_label = "1")
r1 <- smf_calibration_report(doenca$disease[i_te_r], Ptc, bins = 5L, positive_label = "1")
c(ece_raw = r0$ece, ece_cal = r1$ece)
}
TAM_CAL <- c(60L, 120L)
R_REP <- 40L
mc_cal <- lapply(TAM_CAL, function(nc) {
  t(vapply(seq_len(R_REP), function(s) uma_replicata_cal(s, nc), numeric(2)))
})
names(mc_cal) <- as.character(TAM_CAL)
tab_mc <- do.call(rbind, lapply(seq_along(TAM_CAL), function(i) {
  m <- mc_cal[[i]]
  data.frame(n_calibracao = TAM_CAL[i],
             ece_bruto_medio = mean(m[, "ece_raw"]),
             ece_calibrado_medio = mean(m[, "ece_cal"]),
             taxa_melhora = mean(m[, "ece_cal"] < m[, "ece_raw"]))
}))
tab_smf(tab_mc, caption = "Calibração Platt em 40 divisões independentes, por tamanho da part

```

Tabela 3.2: Calibração Platt em 40 divisões independentes, por tamanho da partição de calibração.

n_calibracao	ece_bruto_medio	ece_calibrado_medio	taxa_melhora
60	0,049	0,064	0,300
120	0,046	0,052	0,425

```

ggplot(tab_mc, aes(factor(n_calibracao), taxa_melhora)) +
  geom_col(fill = cores_smf[["primaria"]], width = .5) +
  geom_text(aes(label = fmt(100 * taxa_melhora, 0)), vjust = -0.4) +
  labs(x = "linhas na partição de calibração", y = "taxa de melhora do ECE (%)",
       title = "Calibration improves only with enough data") +
  ylim(0, 1)

```

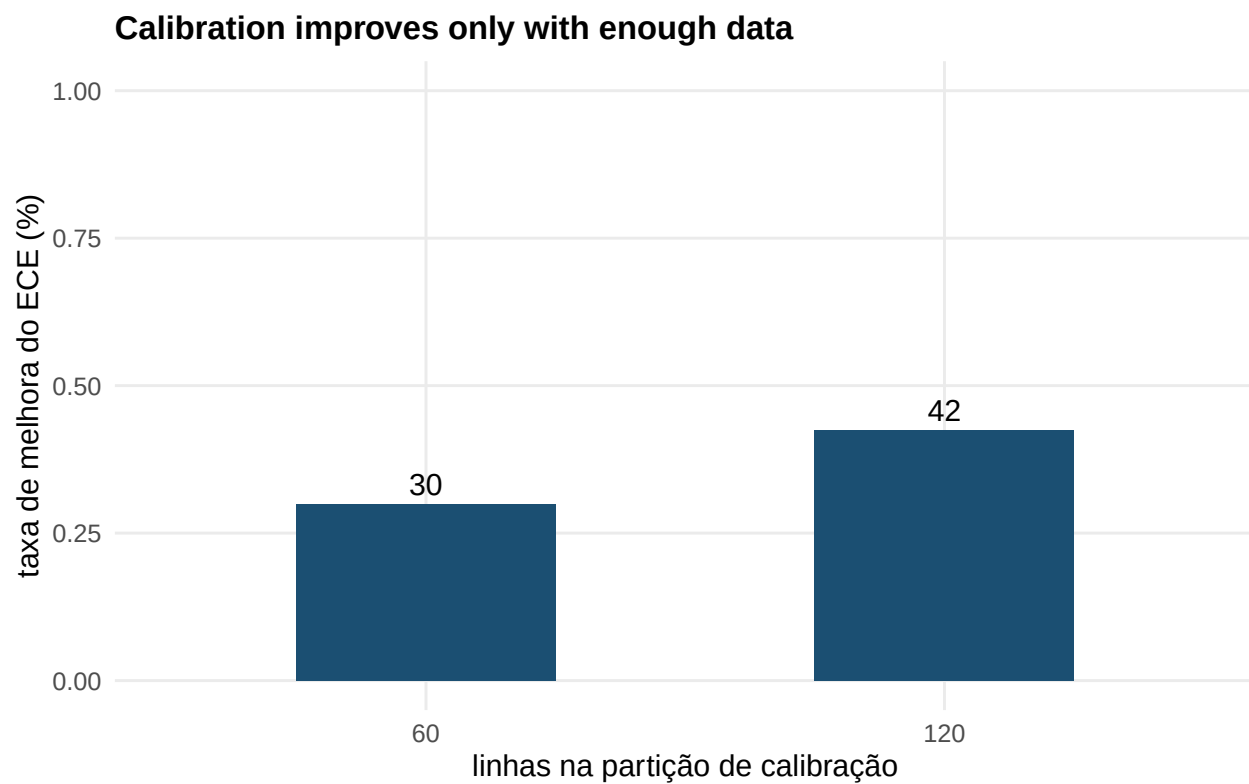


Figura 3.2: A melhora da calibração depende do tamanho da partição: com 60 linhas a taxa de melhora é baixa; com 120, sobe.

4 Limiares: transformar probabilidade em decisão

O limiar de 0,5 raramente é o certo quando classes são desbalanceadas e erros têm custos diferentes. `smf_optimize_threshold()` varre a grade e devolve o melhor limiar para a métrica escolhida — com a curva completa.

```
otimo_f1 <- smf_optimize_threshold(doenca$disease[i_te], p_te, metric = "f1",  
                                  grid = seq(0.05, 0.95, by = 0.05),  
                                  positive_label = "1")  
tab_smf(otimo_f1$curve, caption = "Varredura de limiar para F1 na partição de teste.")
```

Tabela 4.1: Varredura de limiar para F1 na partição de teste.

threshold	value
0,05	0,273
0,10	0,419
0,15	0,479
0,20	0,538
0,25	0,561
0,30	0,543
0,35	0,601
0,40	0,573
0,45	0,582
0,50	0,532
0,55	0,539
0,60	0,539
0,65	0,465
0,70	0,465
0,75	0,465
0,80	0,465
0,85	0,465
0,90	0,465
0,95	0,465

```
ggplot(otimo_f1$curve, aes(threshold, value)) +  
  geom_line(colour = cores_smf[["primaria"]], linewidth = .9) +  
  geom_point(size = 1.8) +  
  geom_vline(xintercept = otimo_f1$threshold, linetype = "dashed",  
             colour = cores_smf[["alerta"]]) +  
  labs(x = "limiar de decisão", y = "F1 no teste",
```

```

title = "Threshold sweep: F1",
subtitle = paste0("Ótimo em ", fmt(otimo_f1$threshold, 2),
                  " (F1 = ", fmt(otimo_f1$value, 3), ")") +
ylim(0, 1)

```

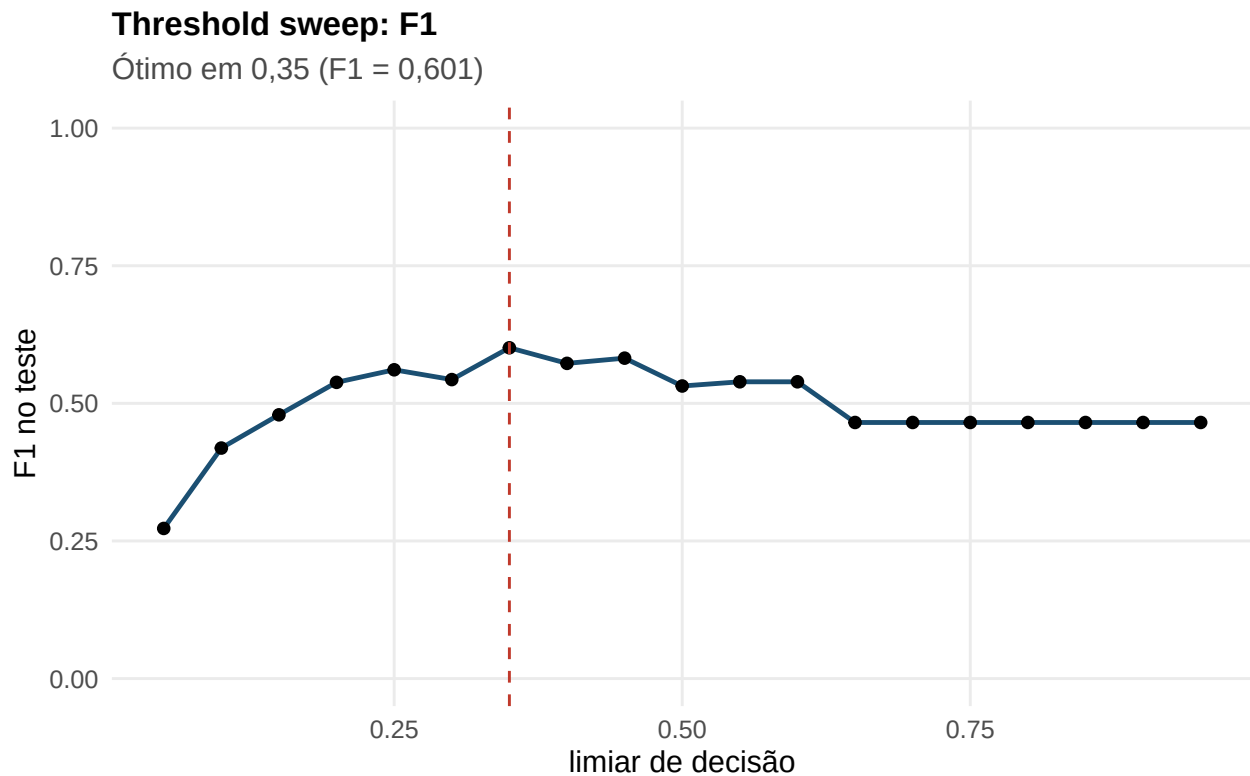


Figura 4.1: Curva de F1 contra o limiar: o ótimo (0,35) é bem menor que o padrão 0,5 — reflexo do desbalanceamento.

💡 No seu experimento

Antes de escolher o limiar, escreva a **matriz de custos**: quanto custa um falso negativo (doença não detectada) contra um falso positivo (controle desnecessário)? Com custo 10:1 para falsos negativos, o limiar ótimo cai ainda mais que o de F1. O pacote não escolhe por você: ele mede a curva para que a decisão seja registrada, não improvisada.

5 Exercícios da Parte 4

Exercício 4.1 — Orçamento de busca

Rode o random search com orçamentos 4, 8 e 16 e registre o melhor RMSE interno e a configuração selecionada. O retorno compensa o custo? Onde você pararia?

Exercício 4.2 — Objetivo ausente

Peça `mae` como objetivo numa especificação que só declara `rmse` e leia a mensagem. Depois conserte a especificação e repita. Por que falhar alto é melhor que devolver archive vazio?

Exercício 4.3 — Calibração com pouca data

Repita a replicata de calibração com 30 linhas. A taxa de melhora cai? Qual é o mínimo de linhas de calibração que você exigiria antes de confiar no calibrador?

Exercício 4.4 — Limiar por custo

Usando a curva de `smf_optimize_threshold()` para sensibilidade, encontre o limiar que garante sensibilidade 0,9. Quantos falsos positivos a mais isso custa em relação ao limiar de 0,5?

Soluções

4.1. O melhor RMSE interno melhora pouco depois de ~8 avaliações neste espaço pequeno; o ganho marginal não paga o custo. O orçamento é decisão de projeto, não de algoritmo.

4.2. O pacote aborta com `TUNING_OBJECTIVE_NOT_IN_METRICS`, listando as métricas da especificação. Falhar alto evita a pior falha: uma tabela de seleção vazia que parece um resultado.

4.3. Com 30 linhas, a taxa de melhora fica próxima de 50% (moeda ao ar). A regra prática: a partição de calibração precisa de centenas de linhas para o calibrador Platt estabilizar; abaixo disso, prefira não calibrar e declarar a limitação.

4.4. Com sensibilidade 0,9 o limiar cai para a faixa de 0,10–0,15, elevando muito os falsos positivos. O ponto pedagógico é que **o limiar é uma decisão de custo**, e precisa vir com a curva que a sustenta.